SVM Classifier

Author: Rohit Mande
Founder and CEO, inttrvu.ai

Website: https://inttrvu.ai

Index

Support Vector Machine

↳ what is support vector

- What is SVM? classification
- Maximum margin hyperplane
- Soft margin classifier
- Kernel trick
- Multiclass classification

What is SVM?

What is SVM?

- Support Vector Machine (SVM) is supervised* learning algorithm
- It is used for building classifiers*
- Once the model is trained on labelled dataset, it can predict new data point belongs to which class
- It is a non-probabilistic binary linear classifier

  0 and 1

  can handle linear datasets
- SVM can perform non-linear classification using kernel trick

* It can be used for regression and unsupervised learning such as clustering too!

SVM → linear problems

SVM + kernel trick → linear and non linear problem

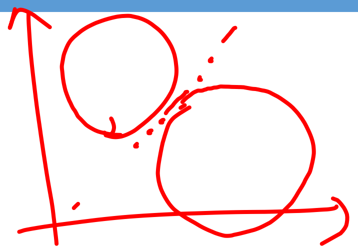
Application of SVM

Applications:

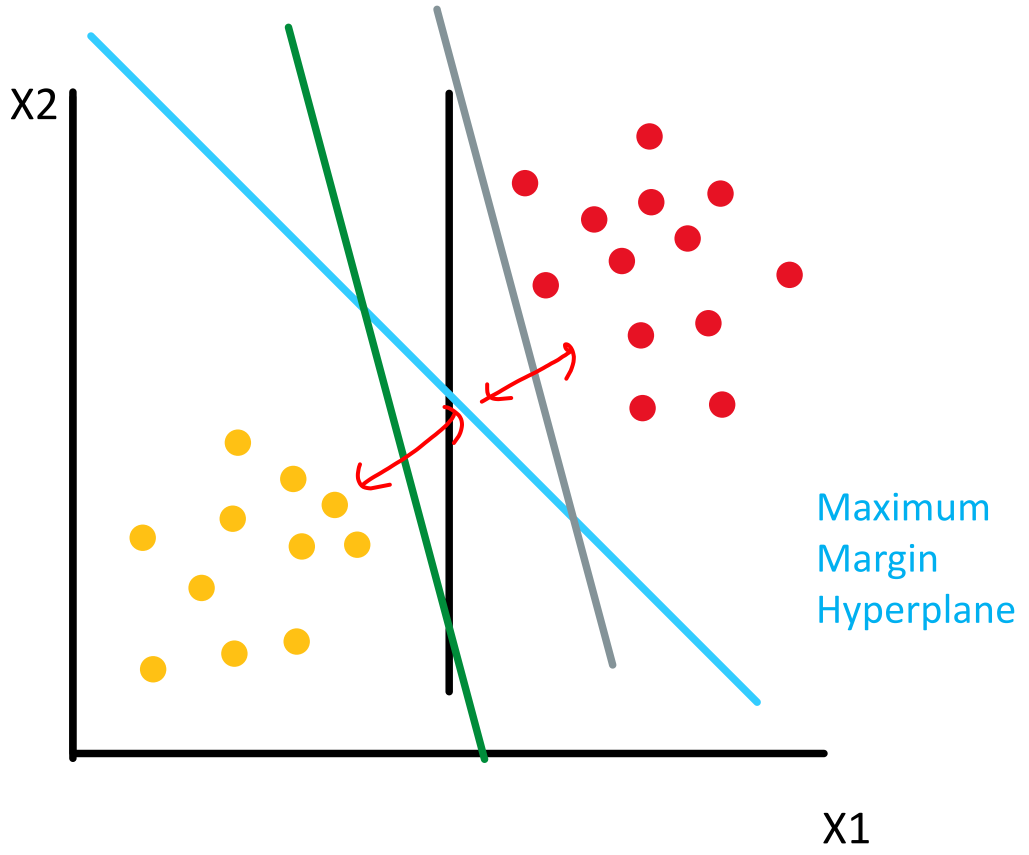
- Largely used in image classification tasks such as object detection (non-linear decision boundary)
- Face detection, Email classification
- Works well on small datasets and has low prediction time than tree-based algorithms (its fast)

Maximum Margin Classfier

Maximum-margin classifier



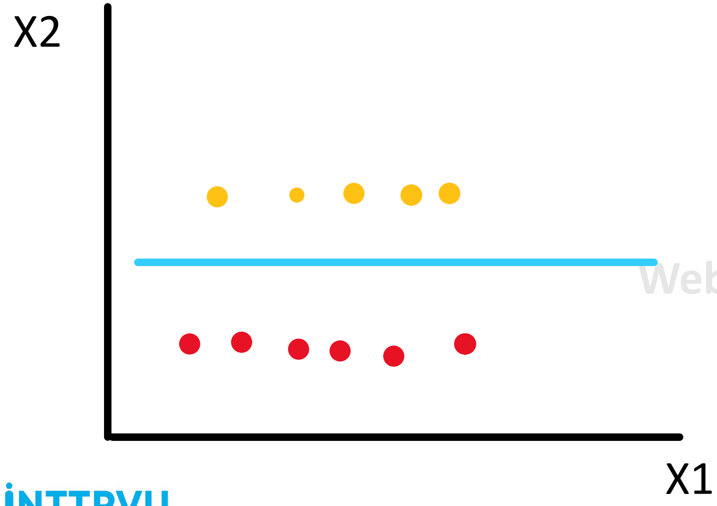
- SVM tries to find a hyperplane which separates two classes with maximum separation

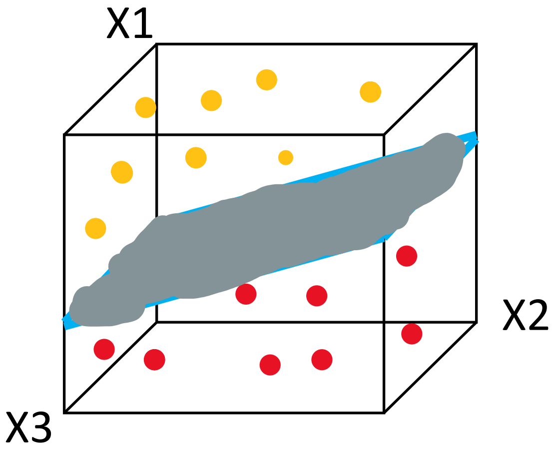


What is Hyperplane?

- Hyperplane is a subspace which has dimeson (d-1) where d is dimension of the data

  For e.g. In 2D space hyperplane is 1D line





Maximum Margin Classifier

Maximum-margin classifier

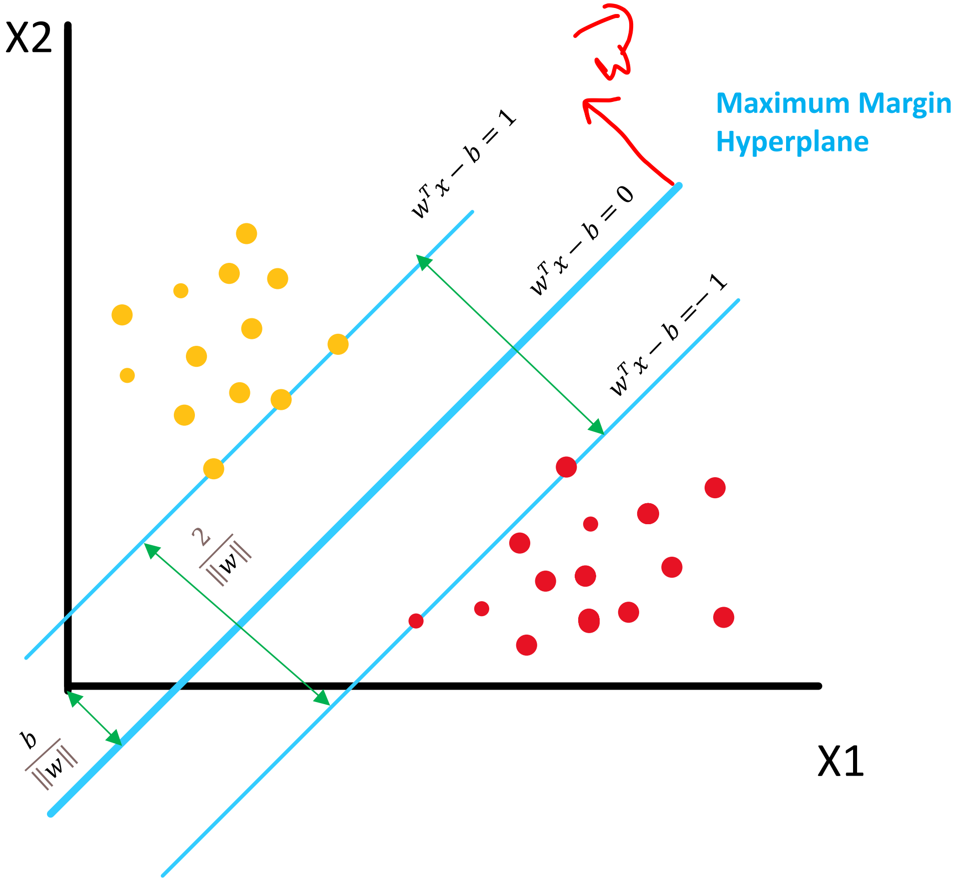
Hyperplane can be written as set of point x satisfying

$$w^T x - b = 0$$

Where $\boldsymbol{w}$ is the normal vector to the hyperplane

$\frac{\boldsymbol{b}}{\|\boldsymbol{w}\|}$ determines the offset of the hyperplane from the origin

Maximum margin = $\frac{2}{\|\boldsymbol{w}\|}$



$\underbrace{w^T}x - \underbrace{b} = 0$

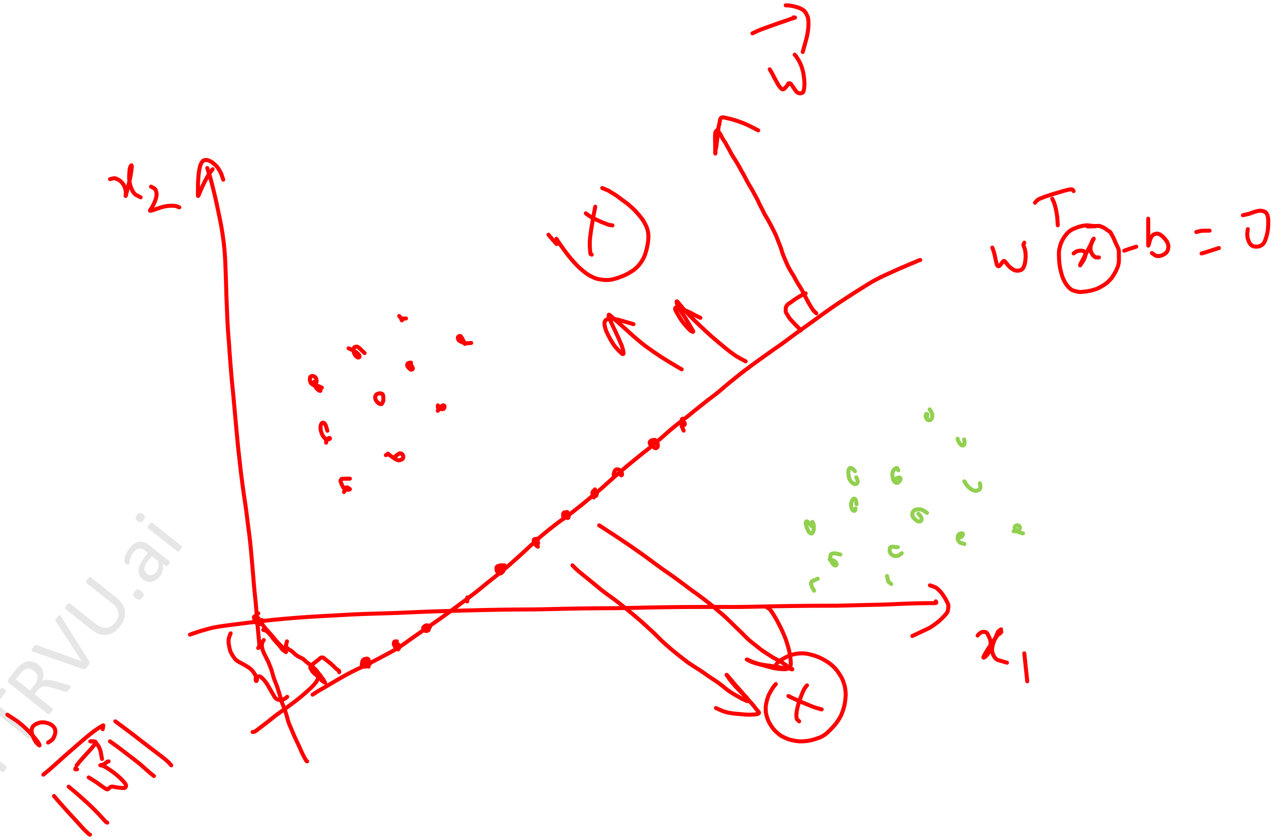
$x_1 + x_2 = 2$ — Eqn of a line

$(x_1 + x_2) - 2 = 0$

$\vec{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ $\quad b = 2$

$w^T = [1 \quad 1]$

$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$

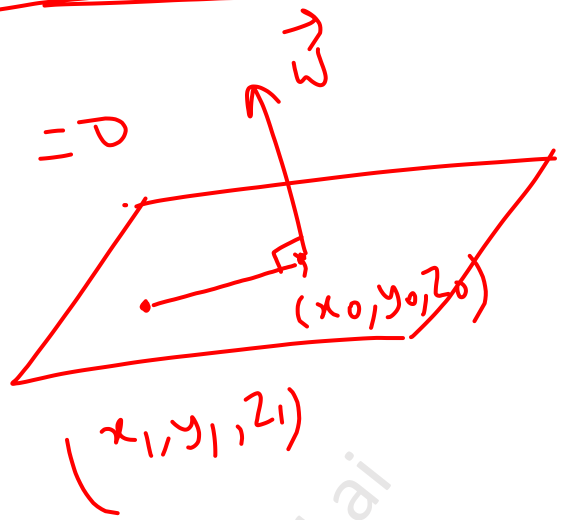


$w^T x - b = 0 \quad = \quad (1 \quad 1)\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} - 2 = \boxed{x_1 + x_2 - 2 = 0}$

OPTIONAL

$w^T x - b = 0$

$$\vec{w} \cdot \left((x_1 - x_0), (y_1 - y_0), (z_1 - z_0)\right) = 0$$



3D plane
⇒ 2D hyperplane

$$\vec{w} = (w_1, w_2, w_3)$$

$$w_1(x_1 - x_0) + w_2(y_1 - y_0) + w_3(z_1 - z_0) = 0$$

$$\underbrace{w_1 x_1 + w_2 y_1 + w_3 z_1}_{w^T x} - \underbrace{\left[w_1 x_0 + w_2 y_0 + w_3 z_0\right]}_{} = 0$$

$$w^T x \quad - b = 0$$

$$w^T x - b = \alpha$$

$$\underbrace{\frac{w^T}{\alpha}}_{w} x - \frac{b}{\alpha} \to b = 1$$

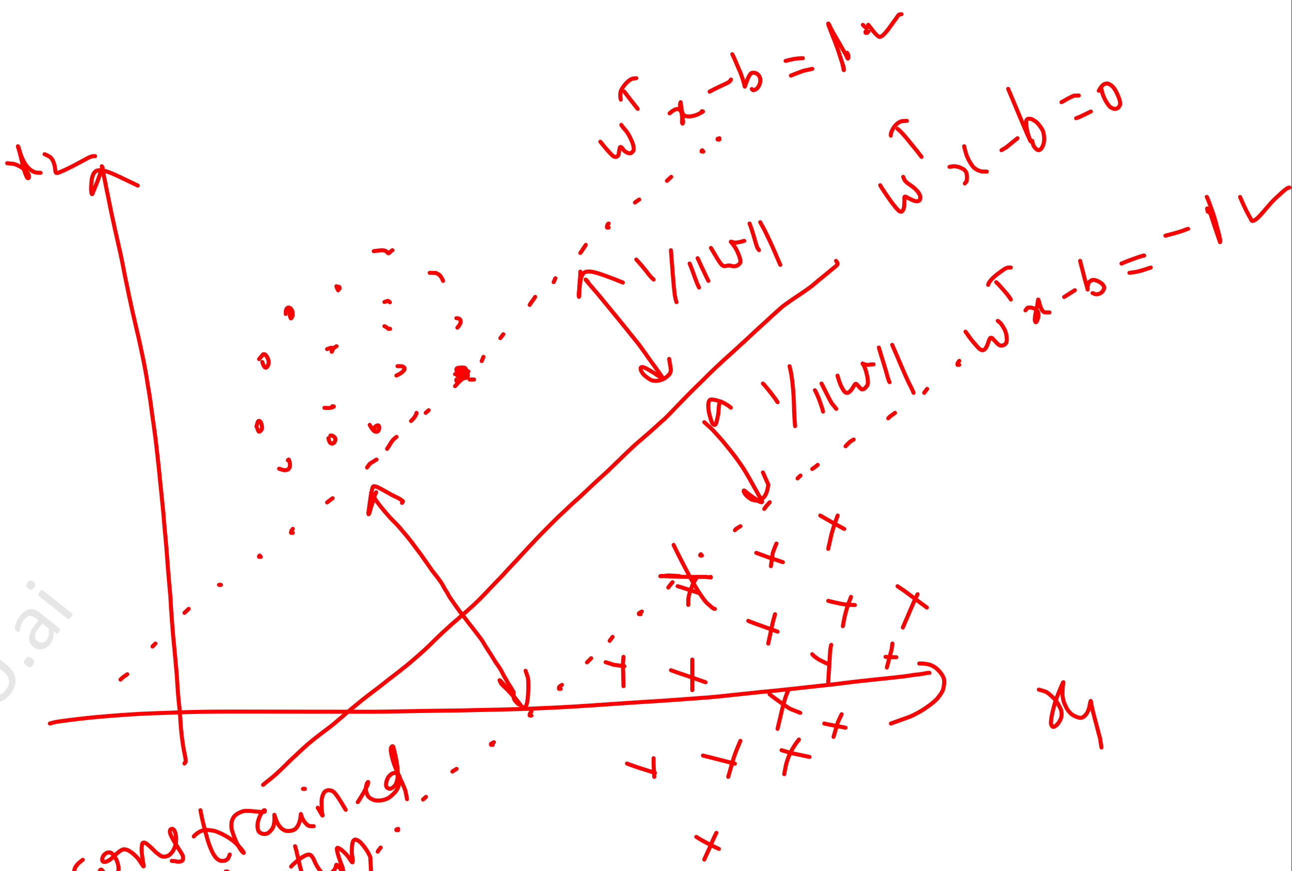
$$w \to \frac{w}{\alpha}$$

$$\text{Margin} = \frac{2}{\|w\|}$$

Maximum margin

$$\boxed{\text{Max } \frac{2}{\|w\|}} \Rightarrow \text{Min } \|w\| \to \text{constrained optimization problem}$$

$$\boxed{\vec{w} = \vec{0}} \times$$

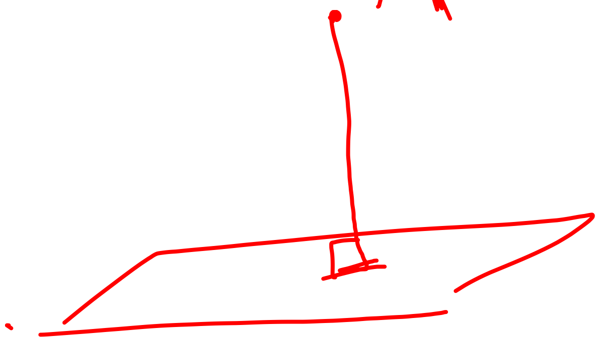


$$\alpha_1 = 1, \quad \alpha_2 = -1$$

$$\frac{|\alpha_1 - \alpha_2|}{\|\vec{w}\|} = \frac{2}{\|\vec{w}\|}$$

Distance of a point from a plane

$X_q \; (x_1, y_1, z_1)$

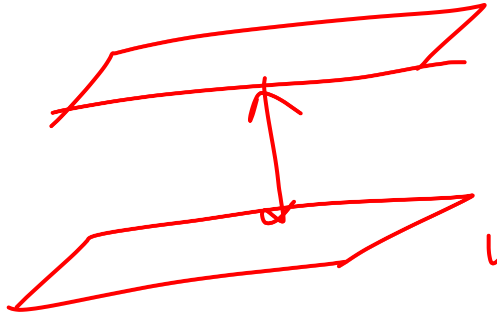


$w^T x - b = 0$

$$\frac{w^T X_1 - b}{\|w\|}$$

Distance from origin $= \dfrac{b}{\|w\|}$

Distance b/w 2 parallel planes



$w^T x - b = \alpha_1$

$w^T x - b = \alpha_2$

$$\frac{|\alpha_1 - \alpha_2|}{\|w\|}$$

Optimization problem

For each data point i

$$\begin{cases} w^T x_i - b \geq 1 & \text{if } y_i = 1 \\ w^T x_i - b \leq -1 & \text{if } y_i = -1 \end{cases}$$

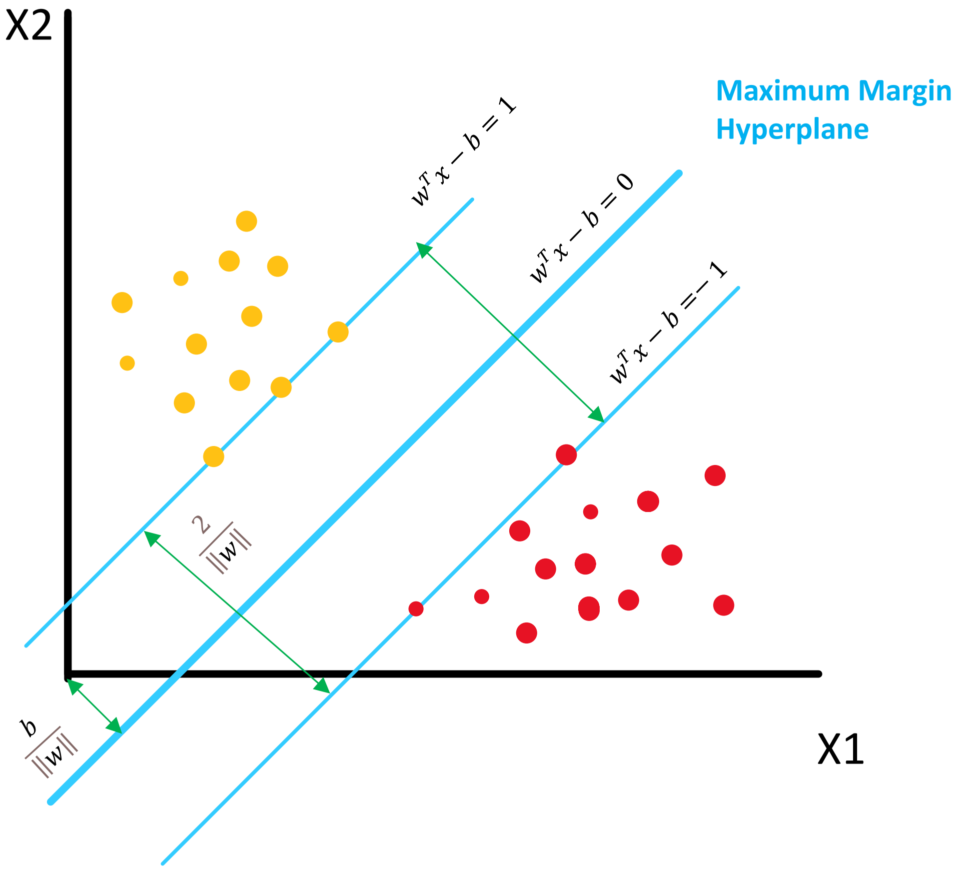
Above equations can be combined as:

$y_i(w^T x_i - b) \geq 1$ for all i≤1≤n

(n is number of data points)

Maximum margin = $\frac{2}{\|\boldsymbol{w}\|}$

To maximize the distance between the planes we must minimize $\|\boldsymbol{w}\|$



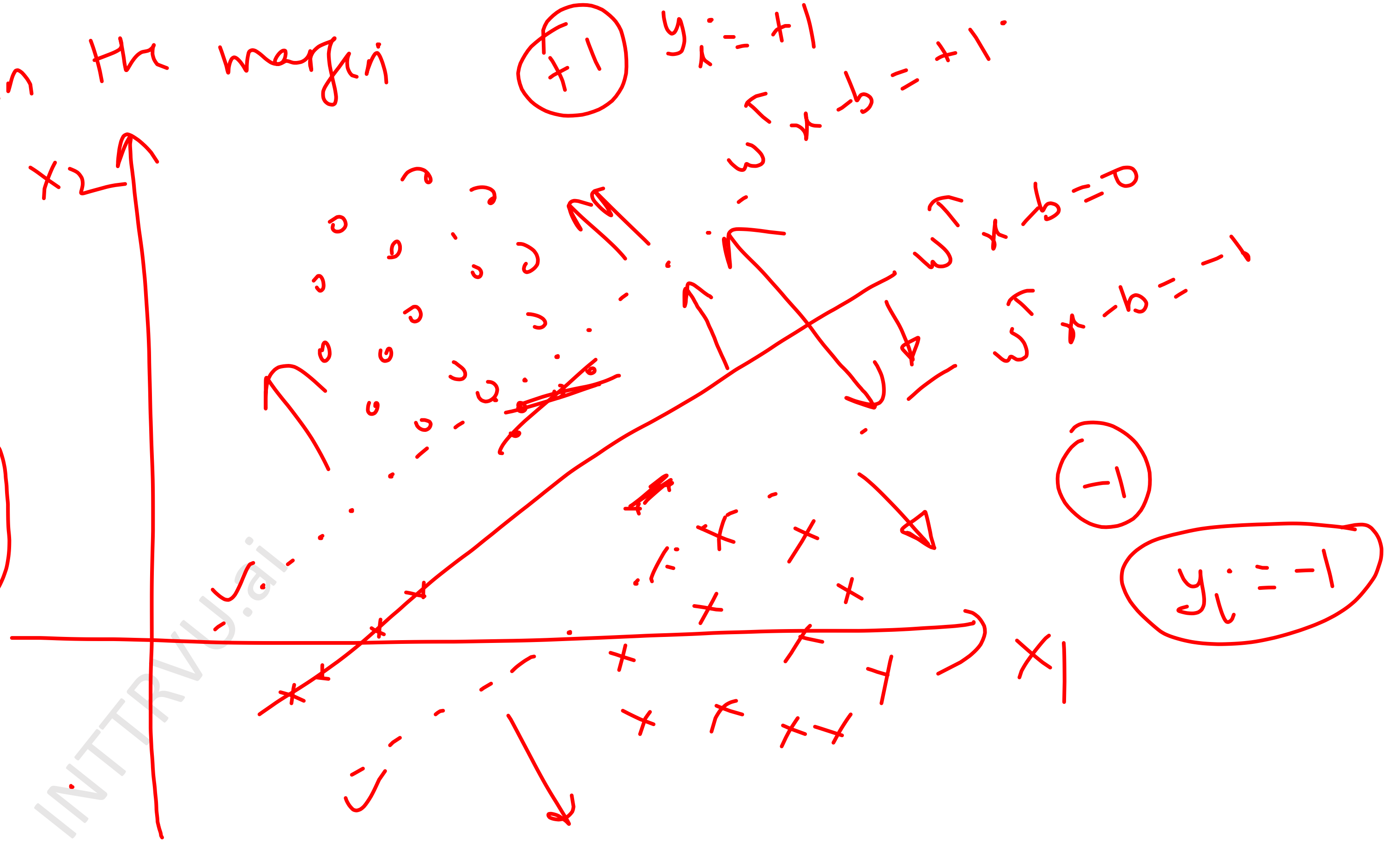
INTTRVU

There are no points lying in the margin

Constraint

I have to minimize $||w||$

such that no point lies in the margin



For $y_i = +1$ ✓

$w^T x_i - b \geq 1$

For $y_i = -1$

$w^T x_i - b \leq -1$

Min $\|\vec{w}\|$ such that no point is allowed in the margin

$y_i = +1$

$w^T x_i - b = 1$

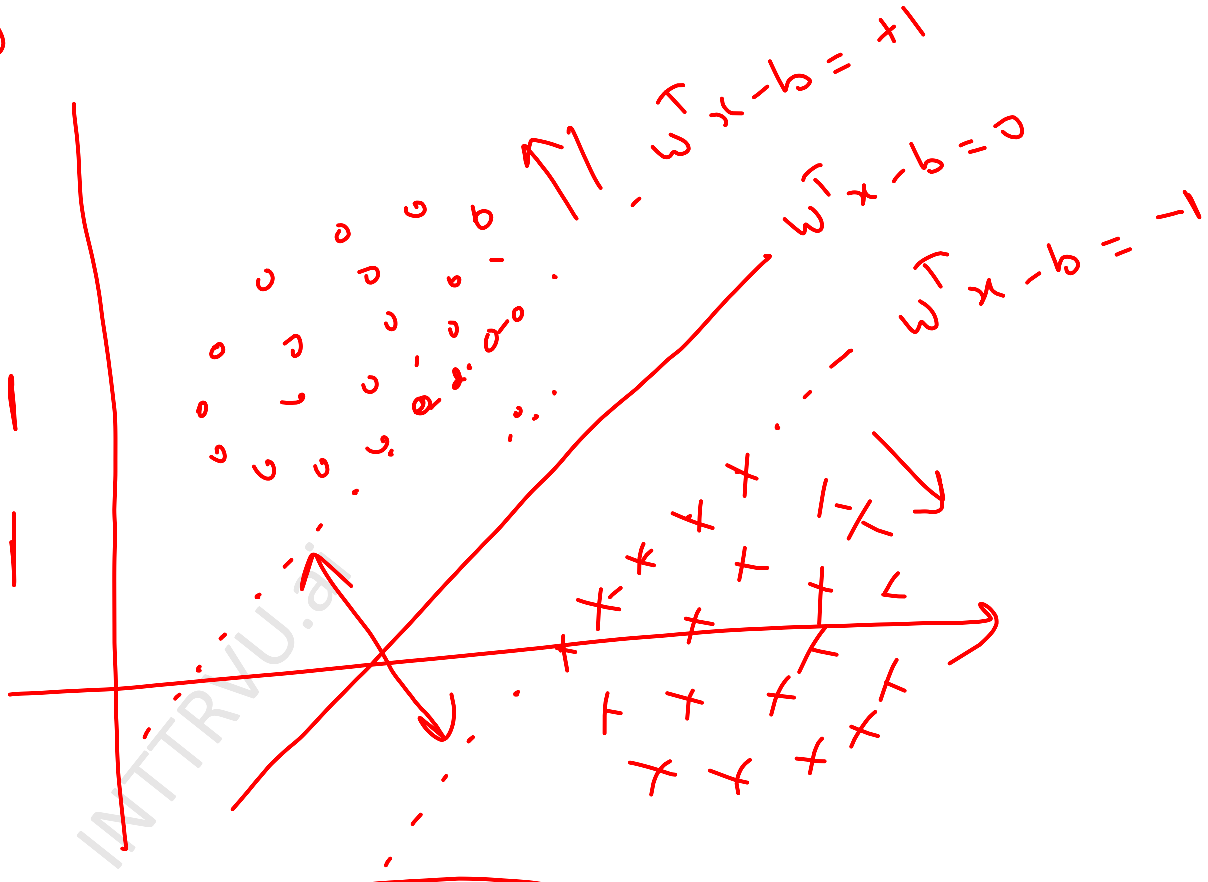
$w^T x_i - b > 1$

$\boxed{w^T x_i - b < 1}$ X

$y_i = -1$

$w^T x_i - b = -1$

$w^T x_i - b < -1$



Min $\|\vec{w}\|$ such that $w^T x_i - b \geq 1$ for $y_i = +1$

and $w^T x_i - b \leq -1$ for $y_i = -1$

For $y_i = +1$

$w^T x_i - b \geq 1$ — (1)

$y_i (w^T x_i - b) \geq y_i$

✓ $y_i (w^T x_i - b) \geq 1$ — (3)

For $y_i = -1$

$w^T x_i - b \leq -1$ — (2)

$y_i (w^T x_i - b) \geq -1 * y_i$

✓ $y_i (w^T x_i - b) \geq 1$ — (4)

$$\boxed{\text{Min } \|w\| \quad \text{such that} \quad y_i (w^T x_i - b) \geq 1}$$

$$\Downarrow$$

$$\boxed{\text{Min } \|w\|^2 \quad \text{such that} \quad y_i (w^T x_i - b) \geq 1}$$

Optimization problem for maximum margin classifier

$$\vec{w} = \sum \alpha_i \vec{x_i}$$

α_i will be non zero for some data points (support vectors)
and zero for others.

$$\vec{w} = \sum \beta_i s_i$$

s_i = support vectors
β_i = coefficients

Optimization problem

Minimize:

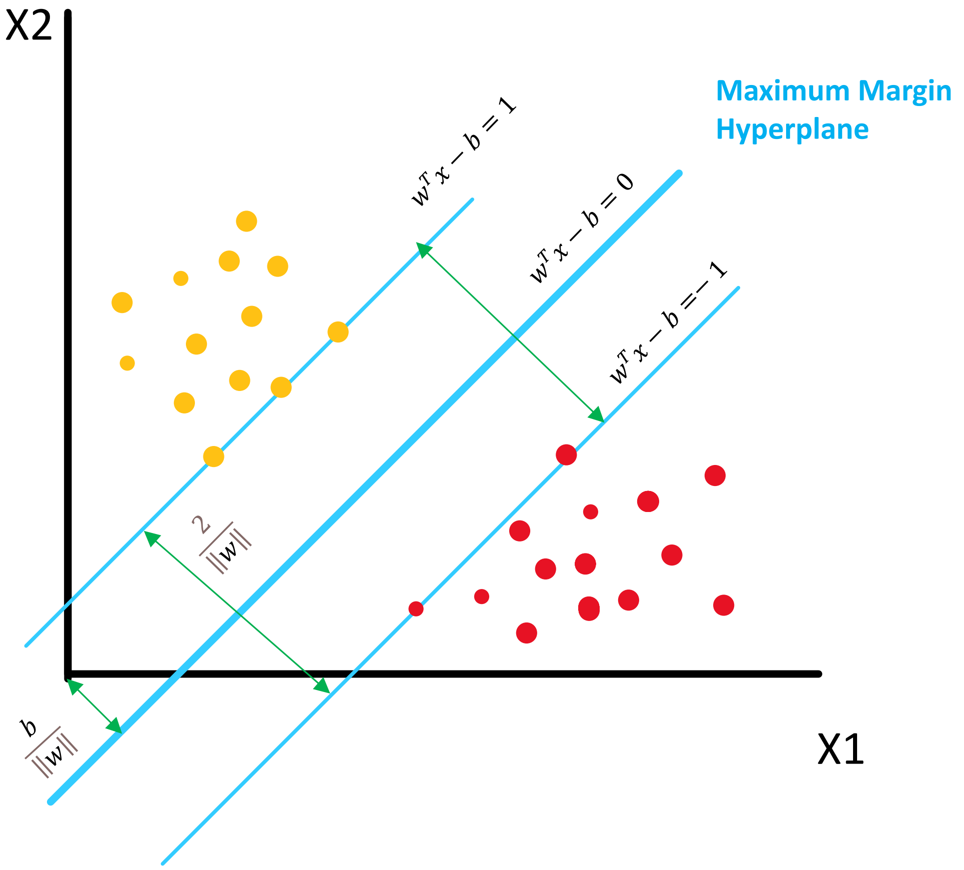
$$\sum_{i=1}^{n} w_i^2 = \|w\|^2$$

optimization problem

Subject to:

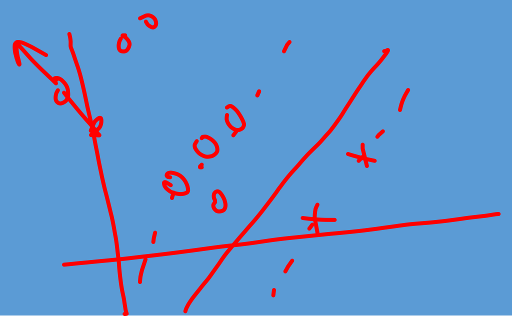
$$y_i(w^T x_i - b) \geq 1$$

Constraints



Support Vectors

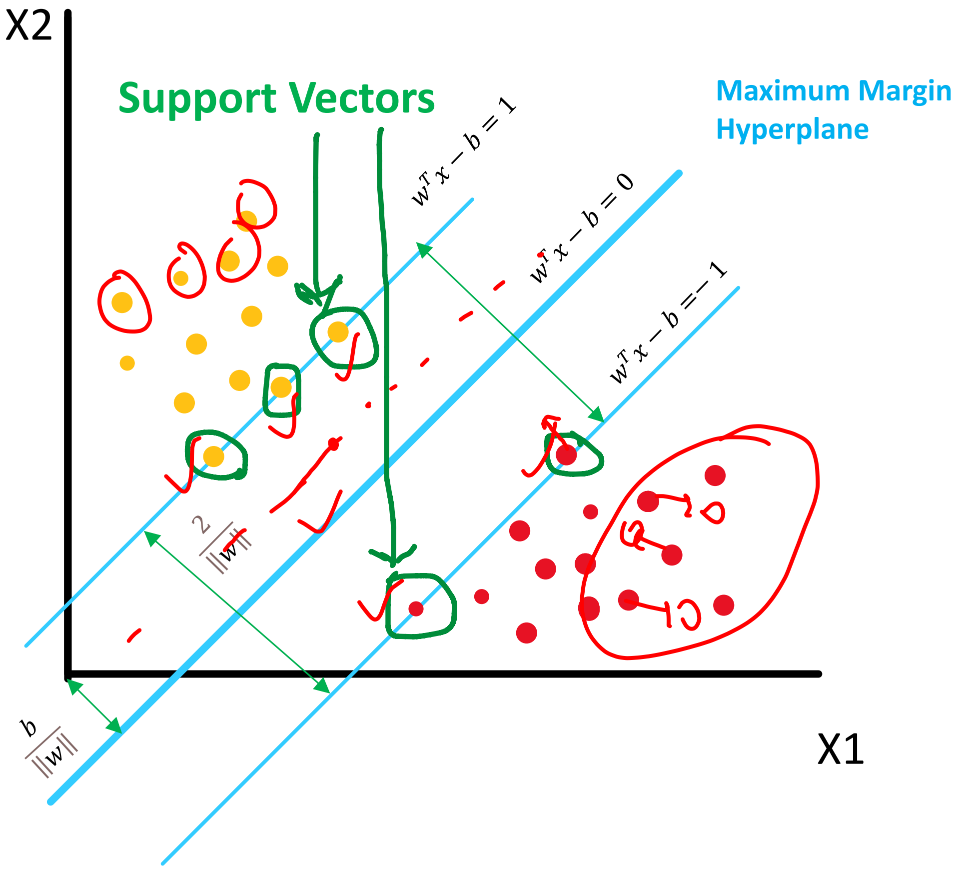
Support Vectors



Support vectors are data points that are closer to the hyperplane

These points influence the position and orientation of the maximum margin hyperplane

Deleting the support vectors will change the position of the hyperplane.



Maximum margin classifier

Hard margin

soft margin

Hard Margin and Soft margin Classifier

Hard Margin Classifier

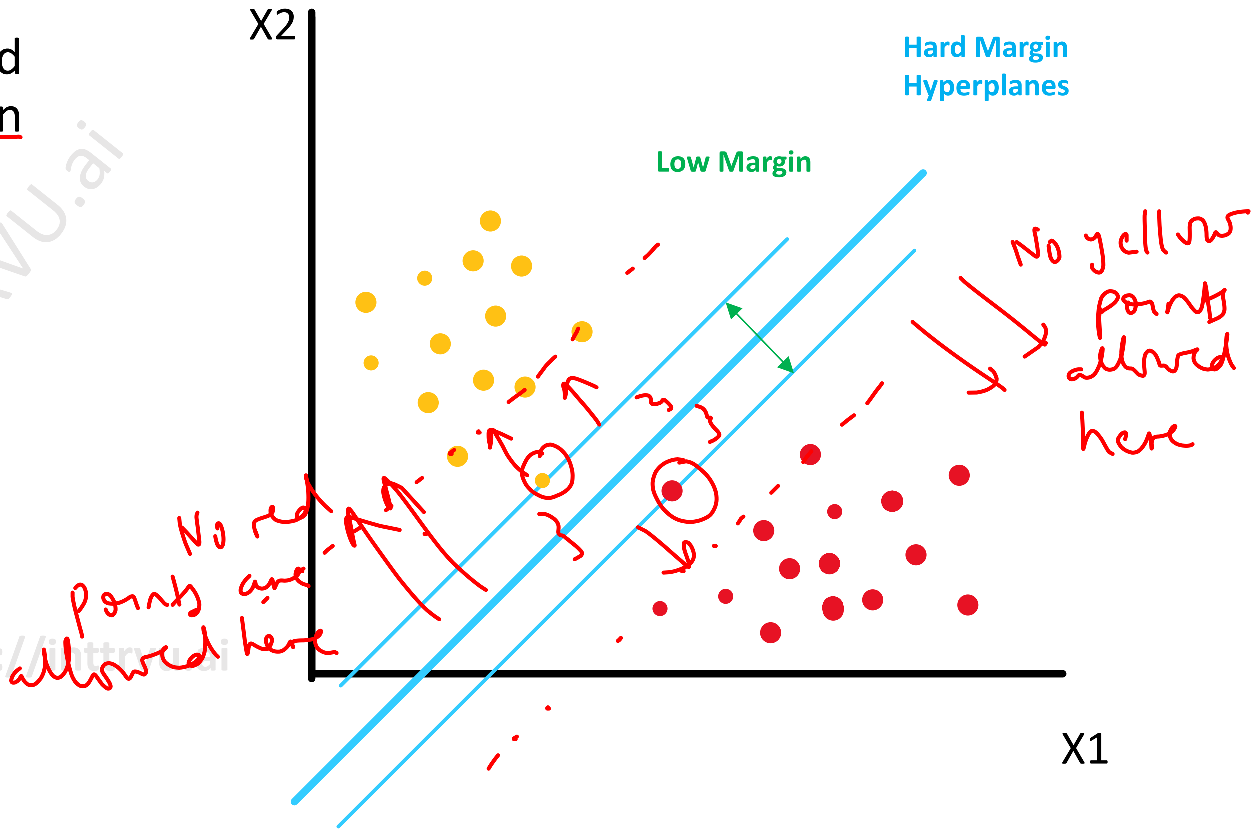
↳ separation is being done perfectly

In hard margin classifier every point in expected to be at the correct side of hyperplane based on its class

This may lead to hyperplanes with small margin

$$\text{Min } \frac{1}{2}\|w\|^2$$

such that $y_i(w^T x_i - b) \geq 1$



Soft Margin Classifier

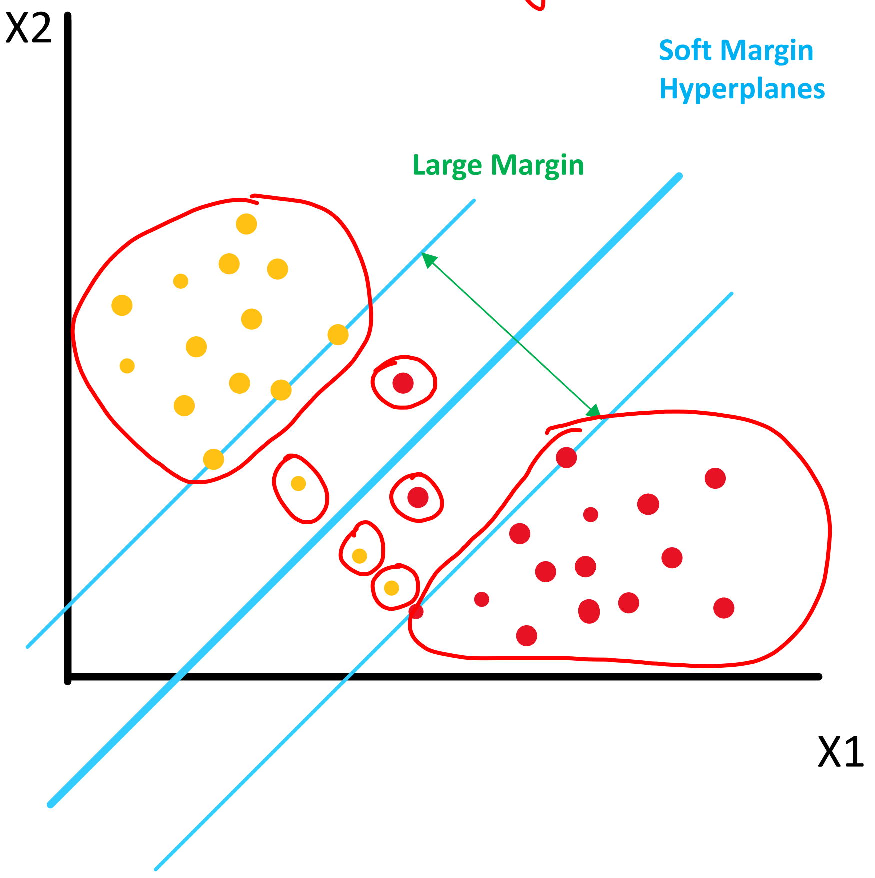
You are allowed to make some errors so that margin can be maximized

In soft margin classifier it is fine to have few points at wrong side of hyperplane – provided it helps in margin maximization

In cases where data is not linearly separable soft margin classifier can be used (if it makes it linearly separable)

Soft margin classifier can lead to hyperplanes with large margin

Non linearly separable



Soft Margin Classifier

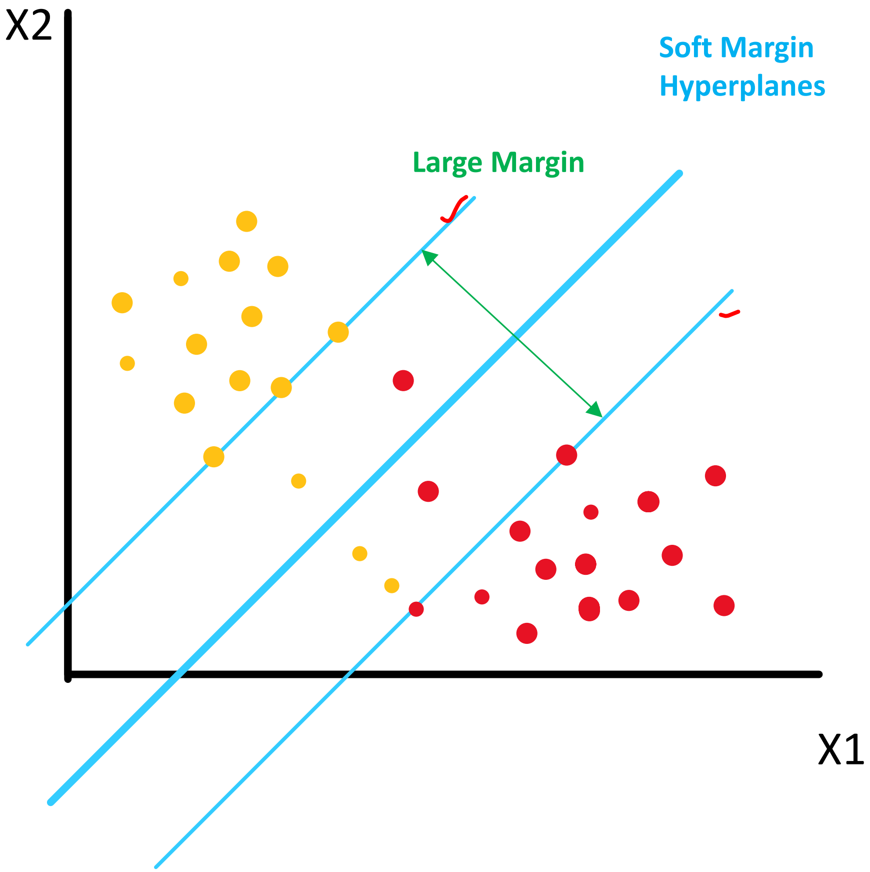
Hinge loss tells you about the distance from the appropriate hyperplane

Hinge loss:

$$\max(0, 1 - y_i(w^T x_i - b))$$

This function is zero if x_i lies on the correct side of the margin.

For data on the wrong side of the margin, the function's value is proportional to the distance from the margin



Hinge Loss for points with $y_i = +1$

$$\max\left[0,\ 1 - \underbrace{y_i(w^T x_i - b)}\right]$$

$i \rightarrow i^{th}$ data point

$R1 \Rightarrow y_i(w^T x_i - b) > 1$

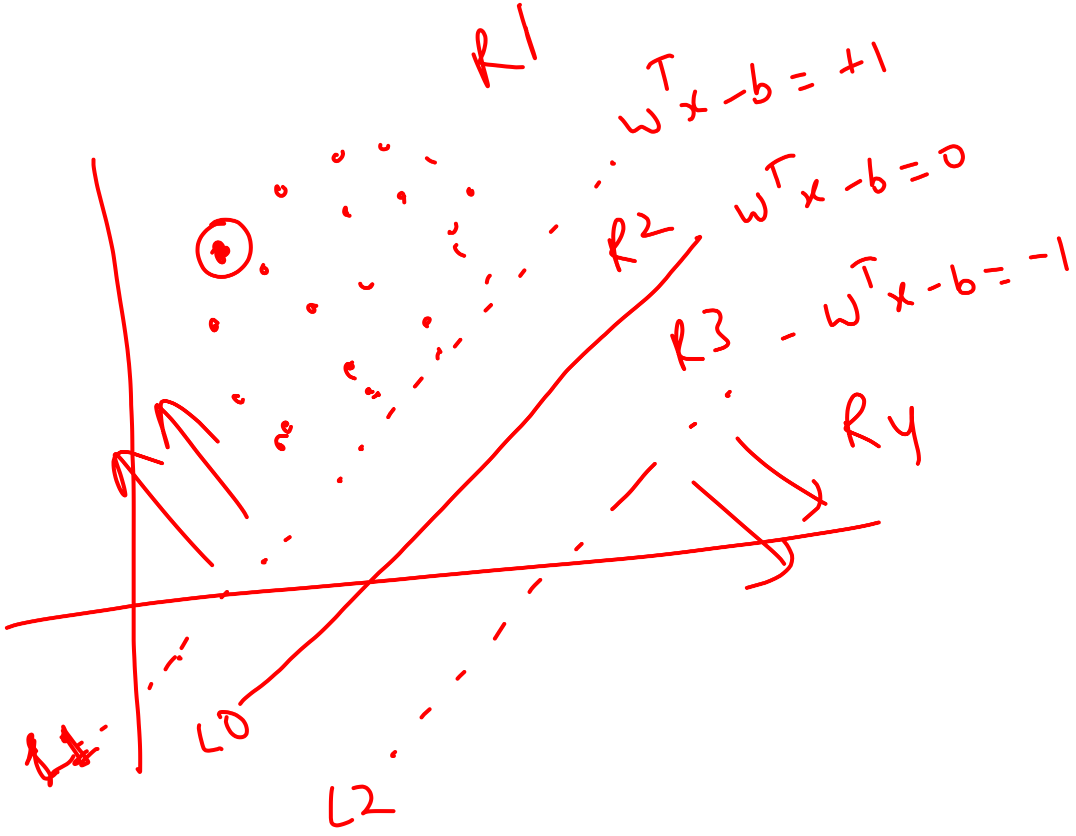
For R_1, Hinge Loss $= \max[0, -ve] = 0$

For $L1$, Hinge Loss $= \max[0, 0] = 0$

For $R2$, Hinge Loss $= ?$ $\quad 0 < y_i(w^T x_i - b) < 1 \Rightarrow$ Hinge Loss

$$= \max\left[0,\ 1 - y_i(w^T x_i - b)\right]$$

$$= 1 - y_i(w^T x_i - b)$$



For L0 $\quad y_i(w^T x_i - b) = 0$

$$\text{Hinge Loss} = \max[0, 1 - y_i(w^T x_i - b)] = \max[0, 1 - 0] = 1$$

For R3 $\quad y_i(w^T x_i - b) \cancel{<} 0 \;,\; y_i(w^T x_i - b) > -1$

$$1 < \text{Hinge Loss} < 2$$

For L2 $\quad \text{Hinge Loss} = 2$

For R4 $\quad \text{Hinge Loss} > 2$

Soft Margin – Optimization Problem

Margin

Hinge Loss

n = Number of data point

$\lambda \rightarrow$ to put a balance between hinge loss and margin

Minimize:

$$\lambda\|w\|^2 + \left[\sum_{i=1}^{n} \max(0, 1- y_i(w^T x_i - b))\right]$$

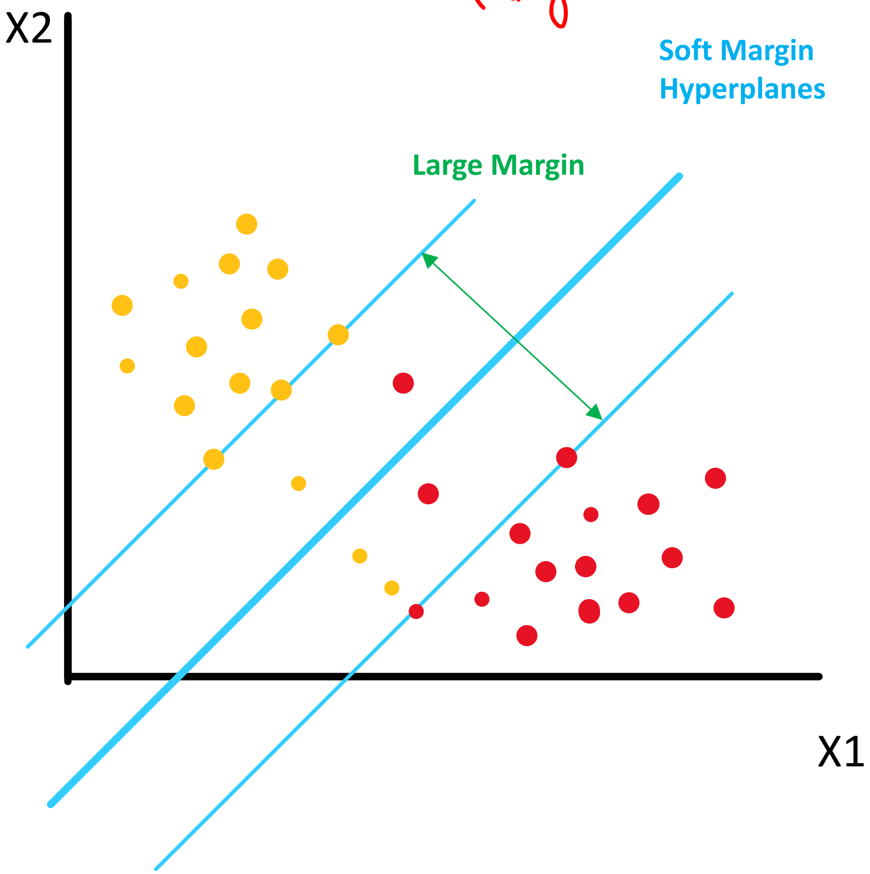
If we multiply by 1/ λ and replace C= 1/ λ

And hinge loss function as ξ_i (error in classification)

$$\|w\|^2 + \underbrace{\frac{1}{\lambda}}_{C}\left[\sum \text{hinge loss}\right]$$

Minimize:

$$\|w\|^2 + C\left[\sum_{i=1}^{n} \xi_i\right]$$



For very large values of C there will be high cost for every misclassification – this may result in hard margin classifier

OPTIONAL

Min $\frac{1}{2}\|w\|^2$ such that $y_i[w^T x_i - b] \geq 1$ $\quad i \in [1, N]$

Lagrange Multipliers

$$L = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{N} \alpha_i \left[y_i \left(w^T x_i - b \right) - 1 \right]$$

$$\frac{\partial L}{\partial w} = 0 \quad ; \quad \frac{\partial L}{\partial b} = 0$$

$$\frac{\partial L}{\partial w} \qquad \vec{w} - \sum_{i=1}^{N} \alpha_i \left(y_i x_i - 0 \right) = 0$$

$$\Rightarrow \boxed{\vec{w} = \sum_{i=1}^{N} \alpha_i y_i \vec{x_i}}$$

$$\frac{\partial L}{\partial b} = 0$$

OPTIONAL

$$L = \frac{1}{2}\|w\|^2 - \sum \alpha_i \left[y_i (w^T x_i - b) - 1 \right]$$

$$\frac{\partial L}{\partial b} = 0 - \sum \alpha_i \left[y_i (0 - 1) - 0 \right]$$

$$\frac{\partial L}{\partial b} = 0 - \sum \alpha_i y_i = 0 \Rightarrow \boxed{\sum \alpha_i y_i = 0}$$

$\boxed{\text{Substitute } \frac{\partial L}{\partial w} \text{ and } \frac{\partial L}{\partial b} \text{ in } L}$

OPTIONAL

$$L = \frac{1}{2}\|w\|^2 - \sum \alpha_i \left[y_i (w^T x_i - b) - 1 \right]$$

$$\|w\|^2 = w^T w$$

$$\vec{w} = \sum_j \alpha_j y_j x_j$$

$$L = \frac{1}{2} w^T w - \sum \alpha_i \left[y_i (w^T x_i - b) - 1 \right]$$

$$L = \frac{1}{2} \left(\sum \alpha_i y_i x_i\right)^T \left(\sum \alpha_j y_j x_j\right) - \sum \alpha_i \left[y_i \left(\left(\sum \alpha_j y_j x_j\right)^T x_i - b\right) - 1 \right]$$

$$L = -\frac{1}{2} \sum \alpha_i \alpha_j y_i y_j \boxed{x_i^T x_j} + \sum \alpha_i$$

only the dot products

Non-linear classifier

No linear hyperplane?

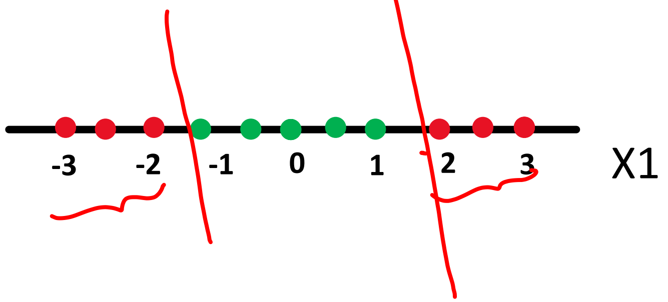
Non linearity can be handled using transformation

If the data cannot be separated by a line hyperplane with original features, we need to transform features such that linear hyperplane can sperate that data

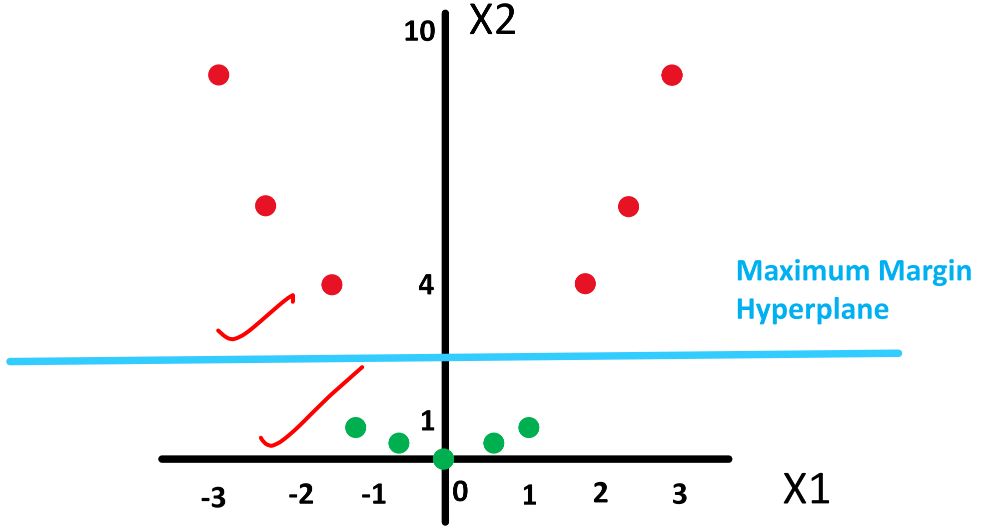
Original data = f(X1)

Transformed data = f(X1,X2)

X2 = $\phi(X1) = X_1^2$



$x_1 \mid x_2 = x_1^2$



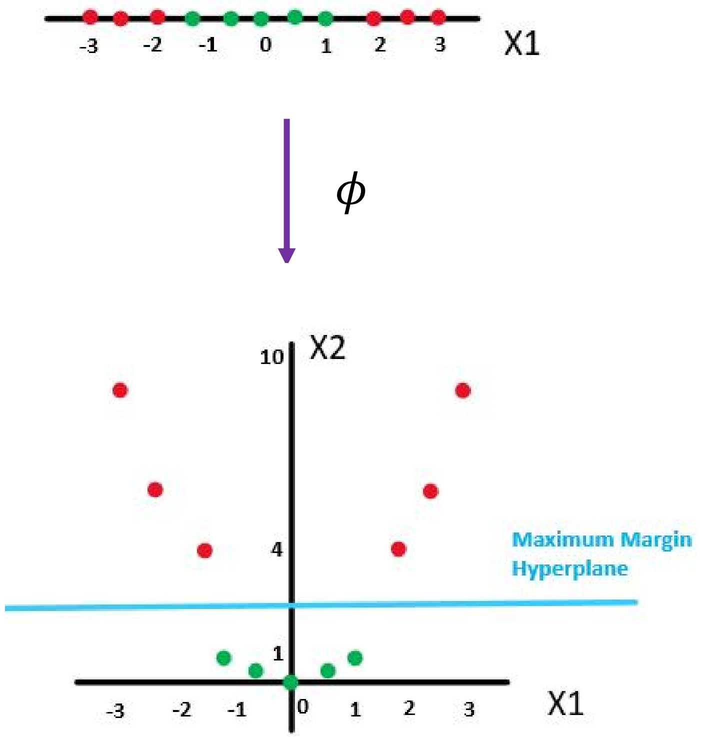
What is Kernel trick?

- Transform data in other space such that linear hyperplane can be found
- The transformation may be nonlinear and the transformed space high-dimensional
- Data is linearly separable in transformed space –> though it has non-linear separation in original space
- Kernel trick helps SVM to build non-linear classifier (classify data which has non-linear separation in original space)

Kernel k corresponds to an inner product in a feature space based on some mapping φ:

$$k(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

Where x_i and x_j are features in original space



Original data → Transformation formula → Transformed data → Train SVM

Skip

Skip

Can I reap the benefits of transformation without actually transforming the data ??

Yes, using kernel trick

We only need the dot products

$$\boxed{x_i^T x_j}$$

$$x_i \xrightarrow{\phi} \phi(x_i)$$

$$x_j \xrightarrow{\phi} \phi(x_j)$$

$$\boxed{\begin{gathered}\phi(x_i)^T \phi(x_j) \\ \| \\ K(x_i, x_j)\end{gathered}}$$

$x_i \rightarrow \phi(x_i)$

$x_j \rightarrow \phi(x_j)$

$\phi(x_i)^T \phi(x_j)$

$$\underbrace{K(x_i, x_j) = \phi(x_i)^T \phi(x_j)}_{\text{Kernel trick}}$$

ϕ is the transformation

K is the dot product calculation as if we have transformed the data

$x_i = \begin{pmatrix} a_i \\ b_i \\ c_i \\ d_i \end{pmatrix}$ $\quad x_j = \begin{pmatrix} a_j \\ b_j \\ c_j \\ d_j \end{pmatrix}$

$x_i^T x_j$

$\begin{pmatrix} a_i \\ b_i \\ c_i \\ d_i \end{pmatrix} \begin{pmatrix} a_j \\ b_j \\ c_j \\ d_j \end{pmatrix}$

$4 \times 1 \quad 4 \times 1$

$\begin{pmatrix} a_i & b_i & c_i & d_i \end{pmatrix} \begin{pmatrix} a_j \\ b_j \\ c_j \\ d_j \end{pmatrix}$

$1 \times 4 \quad 4 \times 1$

Kernels

$$\phi(x_i)^T \phi(x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

(RBF)

Gaussian Radial Basis Function kernel:

$\gamma \downarrow \Rightarrow \sigma \uparrow$

$\gamma \uparrow \Rightarrow \sigma \downarrow$

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

$\frac{1}{2\sigma^2} = \gamma$

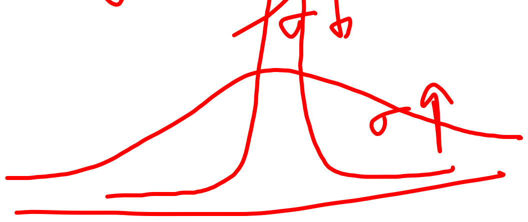
Gamma (γ) defines how far the influence of a single training example reaches, with low values meaning 'far' and high values meaning 'close'.

The model is more likely to generalize well to unseen data for low values of gamma

$\|x_i - x_j\|^2$ is squared Euclidean distance between two feature vectors (rows of the data)

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

σ^2 represents the variance

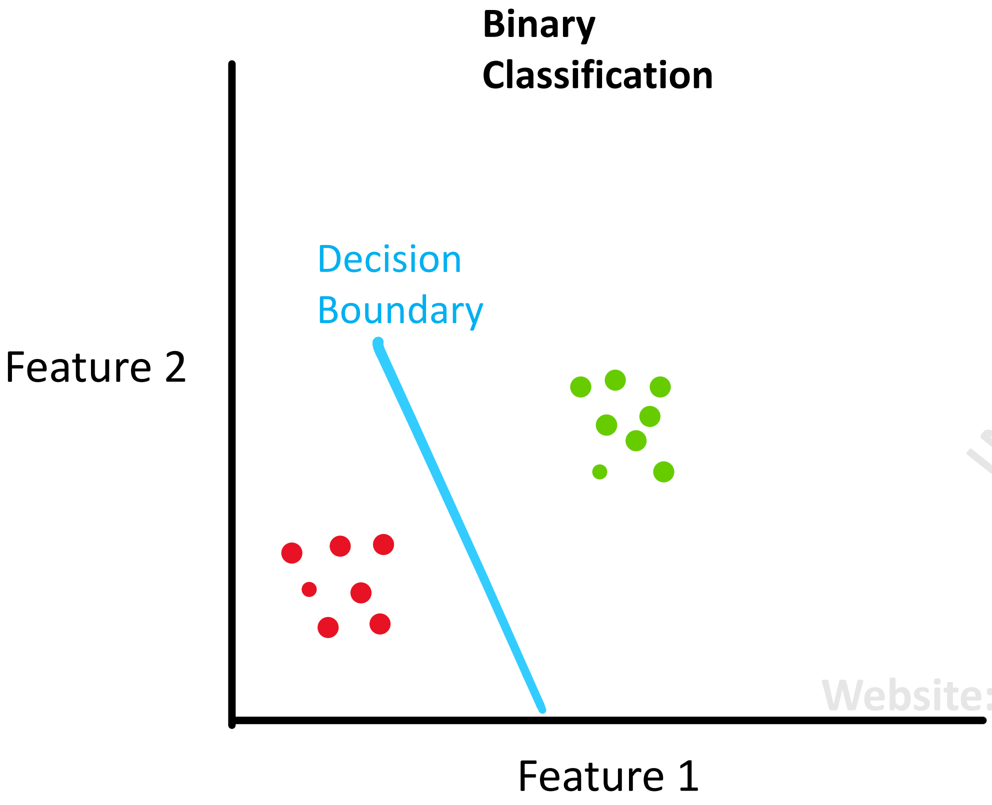


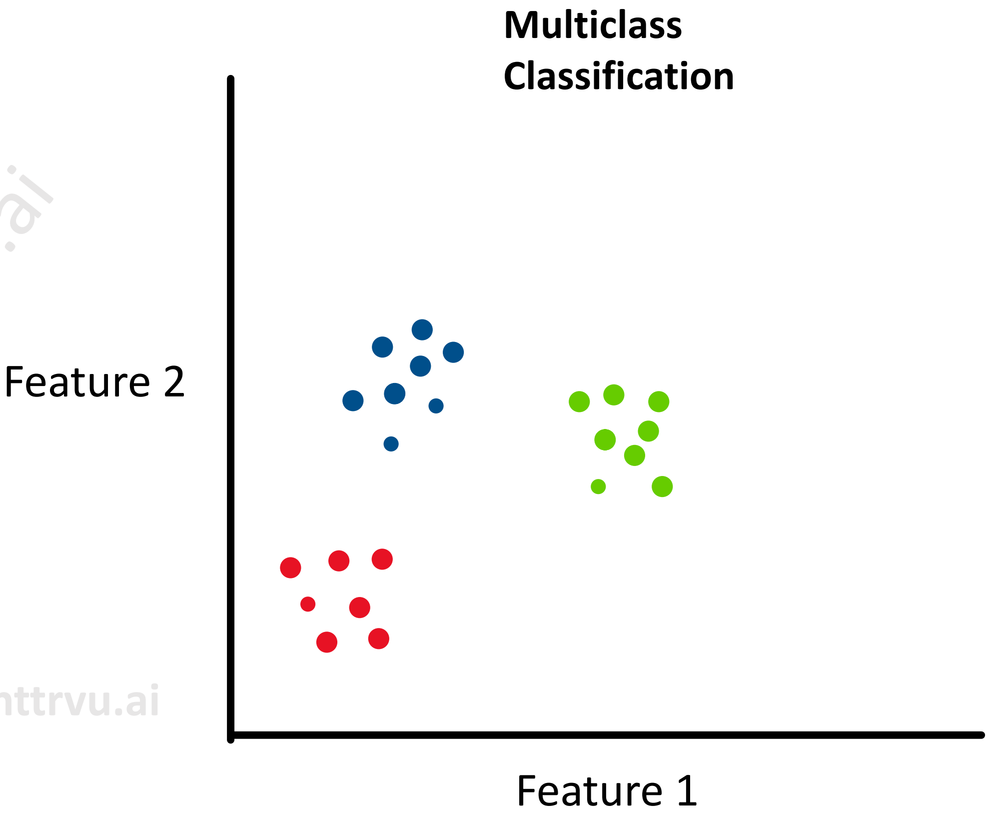
Above function can also be represented by replacing $\gamma = (1/\ 2\sigma^2)$

Note: There are other types of kernels too like polynomial kernel , sigmoid kernel etc.

Multiclass Classification

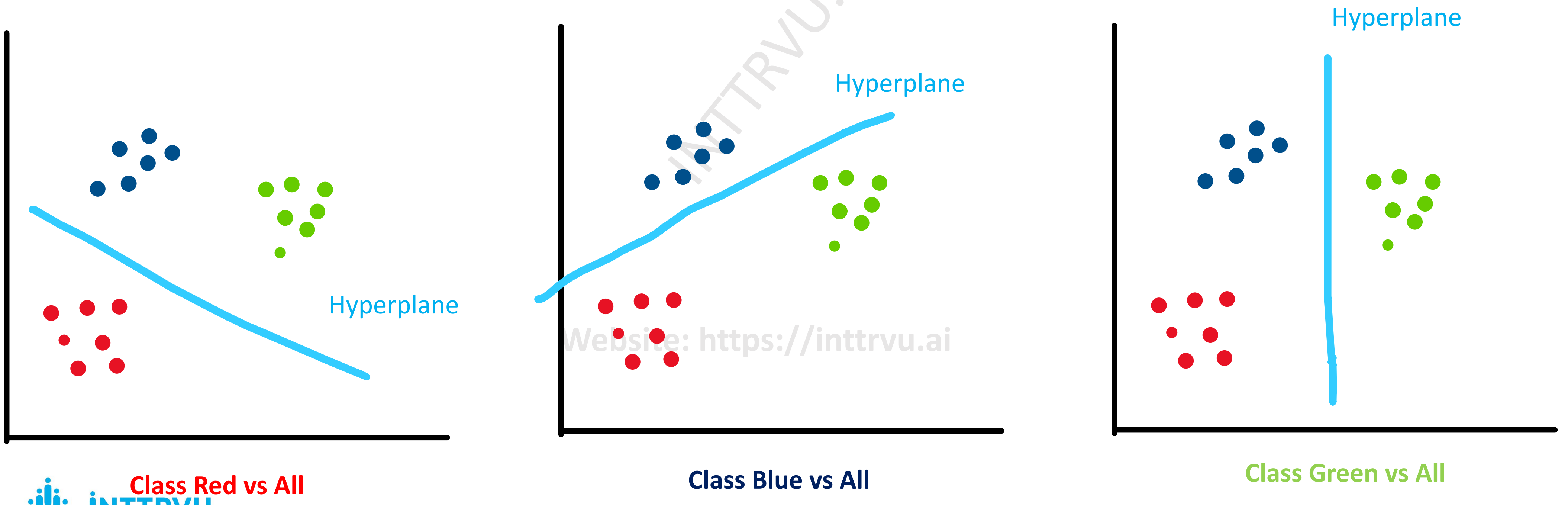
Multiclass classification





Multiclass classification - One vs All

- Train n SVM classifiers where n is number of classes
- Each classifier classifies exactly 1 class vs all other classes (learning n models to separate each class from rest of the classes)



Hyperparameter tuning

Experimentations

Case I Large data available

What we can't do?

You can't tune your hyperparameters on testing data

Train	Test
	85%
	90%
	88%

c = 1

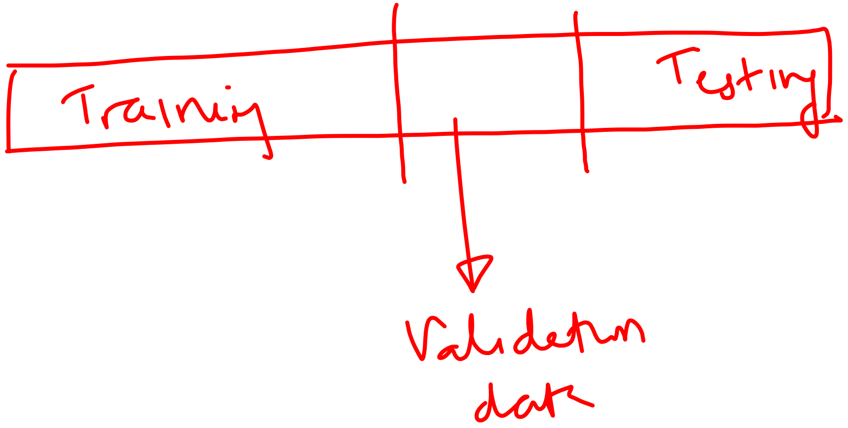
c = 2

c = 3

You have trained your hyperparameter on testing data

You divide data into three parts

1	Training	80	70
2	Validation	10	15
3	Testing	10	15



Case I Large amount of data

	Training	Validation
C=1	~	90%
C=2	~	88%
C=2.5	~	92%

C=2.5
Test and declare

Case 2 Medium amount of data.

K - fold cross validation

Why we need another technique?

Training	80%
Validation	10%
Test	10%

⇒ amount of data available for training will be less.

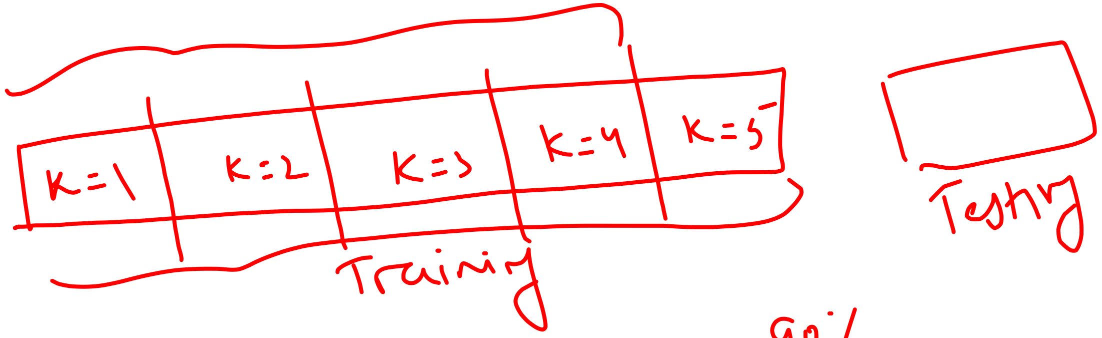
this data is even smaller

⇒ unreliable measurement of your metrics

K- Fold cross validation

We make K- folds in the data

K- folds | cuts



K=5

C=1

Set 1 : Train model on 1 → 4 and validate   K=5 ⇒ acc = 90%.

Set 2 : Train model on 1,2,3,5 and "   K=4 ⇒ acc = 90.5%.

Set 3 : " " " 1,2,4,5 and "   K=3 ⇒ acc : 89.8%.

Set ~~2~~4 : " " " 1,3,4,5 and "   K=2 ⇒ acc : 91%.

Set 5 : " " " 2,3,4,5 and "   K=1 ⇒ acc = 91.5%.

[90, 90.5, 89.8, 91, 91.5]

↳ avg. acc = ___

Step 1 Training and Testing

Step 2 Training data → K fold cross validation

K=5

	Train	Validation	
Set 1	[1,2,3,4	5]	→ Metric1
Set 2	[1,2,3,5	4]	→ Metric2
Set 3	[1,2,4,5	3]	→ Metric3
Set 4	[1,3,4,5	2]	→ Metric 4
Set 5	[2,3,4,5	1]	→ Metric 5

Avg metric value

C=3 → 95% ✓

C=1 → 85%

C=2 → 89%

→ Now train on entire training data

How to decide which values of hyperparameters to tune for?

(1) Random search → [min, max] ; [how many values you want to search for]

; what is probability distribution

uniform

(2) Grid search

↳ you give explicitly the C values

$C \to [1, 1000]$

$c_1 = 5$

$c_2 = 10$

$c_3 = 100$

⋮

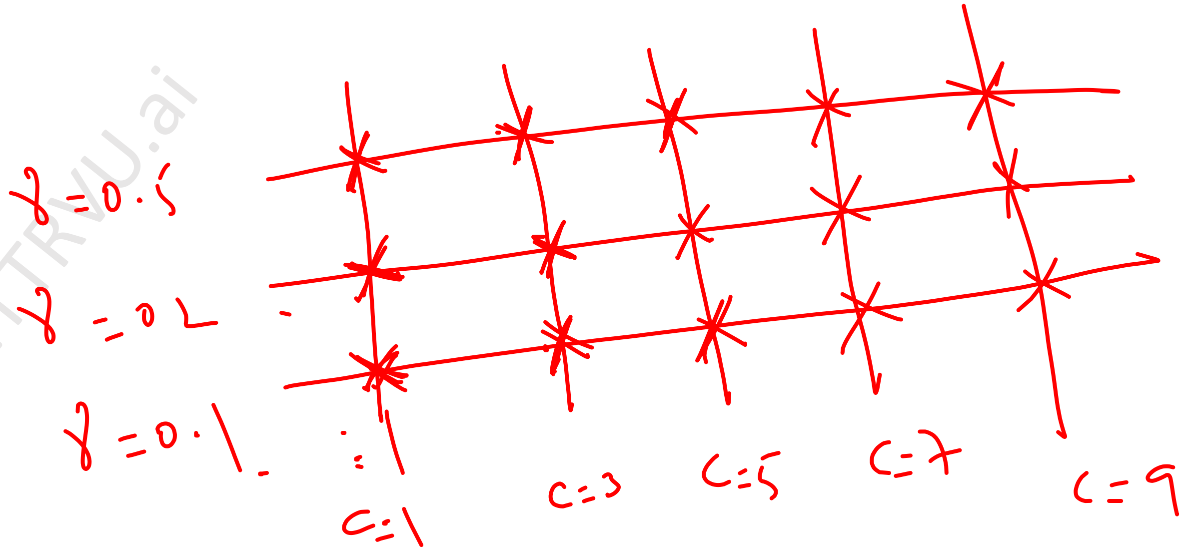
$c_{10} = 9999$

[1, 3, 5, 7, 100]

Grid search

① C = [1, 3, 5, 7, 9]

② Gamma = [0.1, 0.2, 0.5]



Case 3 small data available

N - fold cross validation

N = number of data points available

data

Training

Testing

Questions?